

NEXUS AI

AI / Model Engineering Document

N17–N26 • Models • Routing • Prompts • Context • Benchmarking • Evaluation • Verification • Multi-Agent

Version: 0.1 **Status:** AI Engineering Foundation **Architecture:** Local-first

Purpose: Define how NEXUS AI discovers, selects, invokes, evaluates and coordinates local AI models. The design treats models as interchangeable specialist engines controlled by an orchestration layer rather than as independent chatbots.

USER REQUEST → TASK ANALYSIS → MODEL ROUTING → CONTEXT → MODEL → VERIFICATION → RESPONSE /
AGENT LOOP

1. AI Engineering Vision

NEXUS AI will use a portfolio of local models rather than a single default model. The system will dynamically select models based on task type, model capability, context requirements, resource pressure, benchmark performance and reliability.

The first local runtime is Ollama. The architecture must keep the model gateway independent from Ollama so that another local runtime or provider can be added later without rewriting the orchestration layer.

Engineering Principles

- Capability over model name: route using measured capabilities rather than hard-coded assumptions.
- Local-first inference: core AI processing stays on-device.
- Observable inference: model, latency, context and outcome are measurable.
- Bounded context: only relevant information enters a model prompt.
- Structured outputs where needed: routing, planning and tools use machine-readable contracts.
- Verification for high-impact work: important outputs are checked before completion.
- Graceful fallback: model failure should not automatically fail the entire task.
- Resource awareness: RAM, VRAM, model loading and context size influence routing.
- Versioned prompts and evaluations: changes must be reproducible.

2. N17 — AI Model Strategy

NEXUS treats each installed model as a specialist candidate. Initial role assignments are provisional and must be validated by benchmarking.

Current Model	Initial Role	Candidate Workloads
Qwen2.5-Coder 7B	Coding Specialist	Code generation, debugging, refactoring, code explanation.
DeepSeek-R1 8B	Reasoning Specialist	Planning, complex analysis, decomposition, review.
Qwen2.5 3B	Fast Lightweight	Classification, short summaries, routing helpers, simple tasks.
Llama 3.1 8B	General Assistant	Conversation, writing, summarization, general knowledge.
Qwen2.5-Coder 7B Instruct Heretic	Alternative Coding	Coding comparison, experimentation and benchmark candidate.

Model Role Policy

A model may have multiple capabilities and may serve multiple roles. Role labels are routing metadata, not guarantees. Benchmark results can promote, demote or disable a model for a workload.

Model Lifecycle

Discover → Register → Inspect → Benchmark → Enable → Route → Observe → Re-evaluate → Upgrade/Retire.

3. N18 — Model Registry Specification

The Model Registry is the source of truth for locally available AI models and their routing metadata.

Registry Field	Description
Provider	Inference provider/runtime such as Ollama.
Model ID	Exact model identifier/tag.
Display Name	Human-readable model name.
Version/Tag	Model tag/version used for reproducibility.
Capabilities	General, reasoning, coding, summarization, tool use, etc.
Context Limit	Maximum supported context if known.
Preferred Context	Safe operating range for NEXUS.

Registry Field	Description
Resource Profile	Estimated RAM/VRAM and loading requirements.
Latency Profile	Observed time-to-first-token and throughput.
Quality Scores	Benchmark scores by task family.
Health	Available, loading, busy, unavailable, error.
Routing Weight	Relative preference after capability matching.
Fallback Priority	Order used after primary failure.
Policy Flags	Allowed tasks, tool permissions and safety restrictions.

The registry must support automatic discovery from Ollama and manual metadata overrides where the runtime cannot expose a property reliably.

4. N19 — Model Routing Architecture

Routing is the decision engine that maps a user request to the best available model or model strategy.

Request → intent/task classifier → capability requirements → context estimate → model registry filter → resource check → scoring → selected model → invocation → result quality check → fallback if needed.

Routing Signal	Example Influence
Task type	Coding favors coding models; reasoning favors reasoning models.
Complexity	Simple requests can use smaller/faster models.
Context requirement	Large context may exclude models with insufficient limits.
Resource pressure	Avoid loading another large model when system resources are constrained.
Latency target	Interactive tasks favor fast models.
Quality benchmark	Prefer models with stronger measured performance for the task.
Availability	Unhealthy/unloaded models may be skipped or loaded according to policy.
User preference	Manual model choice can override automatic routing when allowed.

Routing Score

A conceptual score is used rather than a single hard-coded rule: Score = capability fit + benchmark quality + resource fit + latency fit + reliability + user preference. Exact weights are configuration, not fixed business logic.

5. N20 — Model Selection & Fallback Strategy

NEXUS distinguishes between a primary model, preferred fallback and emergency fallback.

Scenario	Primary Behavior	Fallback
Model unavailable	Select next eligible model.	Use fallback by capability class.
Model timeout	Cancel request safely.	Retry with bounded timeout or smaller model.
Context too large	Reduce/compact context if allowed.	Select model with larger context or ask user.
Resource pressure	Avoid new large model load.	Use already loaded/smaller eligible model.
Low-quality output	Run verifier/critic.	Retry with alternative model if policy allows.
Tool-use failure	Keep same model if task state is valid.	Replan or route to another capable model.

Fallback Rules

- Never silently change a user-selected model unless the user has enabled automatic fallback.
- Fallback count must be bounded.
- A failed model invocation must not create duplicate destructive tool actions.
- Fallback decisions must be observable in the task trace.

- Quality fallback and infrastructure fallback are separate reasons and should be logged separately.

6. N21 — Prompt Engineering Architecture

Prompts are versioned engineering assets. NEXUS should avoid building one enormous system prompt for every task.

Prompt Layer	Purpose
Global System	Identity, core principles, output conventions and safety boundaries.
Task System	Task-specific behavior such as coding, research or summarization.
Agent Role	Planner, coder, researcher, reviewer or verifier instructions.
Project Context	Project-specific conventions and goals.
Retrieved Context	RAG evidence and source metadata.
Tool Contract	Available tools, schemas, restrictions and expected output.
User Message	Current user request.
Execution State	Relevant task progress and prior tool results.

Prompt Requirements

Prompts must be versioned, testable, inspectable in debug mode, protected from accidental context injection, and kept within a defined token budget. Tool schemas and retrieved documents must be clearly separated from trusted system instructions.

7. N22 — Context Management Strategy

Context management prevents NEXUS from filling a model context window with irrelevant conversation, memory, files and retrieved documents.

Context Budget → System Instructions + Task + Recent Conversation + Relevant Memory + Retrieved Evidence + Tool State + Output Reserve.

Context Operation	Purpose
Selection	Choose only relevant conversation and memory.
Compression	Summarize older dialogue when necessary.
Retrieval	Fetch evidence only for the current question.
Deduplication	Remove repeated chunks or repeated instructions.
Prioritization	Protect system/policy/task context from lower-priority content.
Budgeting	Reserve tokens for the model's answer or tool call.
Truncation	Remove least-important content first when limits are reached.

Context compression must preserve task constraints, decisions, unresolved issues and source references. It must never silently discard security or permission information.

8. N23 — Model Benchmarking Plan

Benchmarking establishes evidence for routing decisions. NEXUS should maintain a small repeatable local benchmark suite rather than relying on subjective impressions.

Benchmark Family	Example Test
General Chat	Instruction following and coherent response.
Reasoning	Multi-step logical problem.
Coding	Implement a small function with tests.
Debugging	Identify and repair a controlled bug.
Summarization	Compress a known document while retaining facts.
RAG	Answer from provided evidence and cite it.
Structured Output	Return valid JSON against a schema.

Benchmark Family	Example Test
Tool Planning	Choose appropriate tools and arguments.
Safety	Refuse or escalate prohibited/high-risk tool behavior.
Latency	Time-to-first-token and generation throughput.

Benchmark Outputs

For each model: score, latency, token throughput, resource use, error rate, context behavior and qualitative notes. Benchmark results must include model tag/version and benchmark-set version.

9. N24 — AI Evaluation Framework

Evaluation measures whether NEXUS is producing useful, grounded and reliable results. Evaluation is divided into model quality, system quality and task completion.

Evaluation Layer	Measures
Model	Instruction following, reasoning, coding, summarization, structured output.
RAG	Retrieval relevance, context recall, citation correctness, groundedness.
Agent	Plan quality, tool selection, task completion, recovery behavior.
System	Latency, failures, resource usage, stability.
Safety	Permission adherence, tool boundary compliance, prompt injection resistance.

Evaluation Modes

Offline evaluation uses a fixed test set. Regression evaluation runs after changes to prompts, routing or models. Online/local telemetry measures real task outcomes with privacy-aware logging.

Core Metrics

Task Success Rate, Grounded Answer Rate, Citation Accuracy, Retrieval Recall@K, Retrieval Precision@K, Tool Success Rate, Agent Completion Rate, Fallback Rate, Model Error Rate, Time to First Token, End-to-End Latency and Resource Utilization.

10. N25 — AI Verification & Critic System

The verifier checks outputs when the task warrants additional validation. It must not become an unnecessary second model call for every simple message.

Verification Type	Example
Grounding Check	Does the answer agree with retrieved evidence?
Citation Check	Do cited sources support the claims?
Schema Check	Is structured output valid?
Code Check	Does generated code pass configured tests/lint?
Tool Result Check	Did the tool actually produce the requested state/result?
Constraint Check	Did the answer follow explicit user/task constraints?
Safety Check	Is a proposed action within policy?

Verification Outcomes

PASS → return result. SOFT_FAIL → revise or request clarification. RETRY → rerun with bounded alternative strategy. BLOCK → prevent action and explain the constraint.

For coding tasks, execution-based verification is preferred over another language-model opinion whenever safe tests are available.

11. N26 — Multi-Agent Architecture

Multi-agent behavior is introduced only after the single-agent runtime, tools and verification layers are stable.

User Goal → Planner → Specialist Agent(s) → Tool/Knowledge Actions → Reviewer/Verifier → Final Synthesizer.

Agent Role	Primary Responsibility	Typical Model Candidate
Planner	Decompose task and define bounded steps.	DeepSeek-R1 8B
Researcher	Retrieve and synthesize knowledge.	Llama/Qwen depending on benchmark
Coder	Create or modify code.	Qwen2.5-Coder 7B
Tester	Run/analyze tests and report failures.	Coder model + tools
Reviewer	Inspect result against requirements.	Reasoning/general model
Verifier	Check evidence, constraints and completion.	Best benchmarked verification model

Agent Contract

Every agent receives a task, constraints, available tools, context budget, completion criteria and stop conditions. Agents return structured status: plan, actions, observations, result, confidence/verification status and unresolved blockers.

Multi-Agent Guardrails

- Maximum steps per task.
- Maximum retries per step.
- Maximum tool calls per task.
- Explicit shared-state schema.
- No agent may bypass tool permissions.
- No circular delegation without a hard stop.
- Human approval for destructive/high-impact operations.
- Final verifier required for tasks marked high-impact.

12. Model-to-Agent Architecture

NEXUS separates model identity from agent identity. An agent is a role with instructions, tools and success criteria; a model is the inference engine selected to perform that role.

Agent Role → Capability Requirements → Model Router → Selected Model → Model Gateway → Agent Runtime.

This allows the same Planner agent to use different models depending on available hardware, benchmark results or user policy.

13. Resource-Aware AI Runtime

Because the target environment may have limited RAM/VRAM, NEXUS must treat resource availability as a routing signal.

Resource Signal	System Action
Low VRAM	Prefer smaller/loaded models; avoid concurrent large loads.
High RAM pressure	Reduce context and concurrency; unload inactive models if safe.
Model load contention	Queue or serialize model loading.
Long context	Use compaction/retrieval and reserve output tokens.
High latency	Offer faster model or reduce task complexity where appropriate.
GPU unavailable	Fall back to CPU-capable configuration if acceptable.

14. Observability & Traceability

Every significant AI task should produce a trace containing task ID, selected model, routing reason, prompt/template version, context sources, tool calls, latency, token/resource metrics and verification outcome, subject to privacy/redaction rules.

A user-facing advanced view should show a concise execution summary rather than exposing hidden chain-of-thought. NEXUS can expose tool activity, sources, model identity, status and verification evidence without revealing private internal reasoning.

15. AI Security Requirements

- Treat retrieved documents and tool outputs as untrusted content.
- Separate trusted system/policy instructions from external content.
- Validate structured model outputs before execution.
- Never execute raw model text as a command.
- Use allowlisted tools and scoped arguments.
- Detect and handle prompt injection in documents and tool results.
- Redact secrets from telemetry.
- Require confirmation for destructive actions.

16. AI Engineering Quality Gates

Gate	Required Condition
Model Integration	Model can be discovered, invoked and health-checked.
Routing	Routing decision is deterministic/reproducible for the same configured inputs.
Context	Context stays within budget and preserves required constraints.
RAG	Retrieved evidence retains provenance.
Tools	All tool calls pass validation and policy.
Evaluation	Benchmark suite exists for every routed task family.
Verification	High-impact workflows have explicit verification.
Fallback	Failure recovery is bounded and observable.
Agents	Agent loops have limits, stop conditions and shared-state contracts.
Privacy	Core workflows can operate locally without mandatory cloud calls.

17. Implementation Order for N17–N26

1. Model adapter + Ollama connection → 2. Model registry → 3. normalized model gateway → 4. basic manual model selection → 5. task classifier/router → 6. context manager → 7. benchmark harness → 8. evaluation store → 9. verifier → 10. planner/agent runtime → 11. specialist agents → 12. multi-agent coordination.

Do not start with autonomous multi-agent behavior. Prove single-model invocation, routing, context and evaluation first.

18. Final AI / Model Engineering Statement

NEXUS AI's intelligence is an engineered system rather than a single model. The product value comes from combining local model specialization, intelligent routing, controlled context, RAG, memory, tools, evaluation and verification. This architecture allows NEXUS to improve as new local models become available without redesigning the application.